Journal of Computer and System Sciences 79 (2013) 530–541

Contents lists available at SciVerse ScienceDirect

Journal of Computer and System Sciences

www.elsevier.com/locate/jcss

Feasibility interval for the transactional event handlers of P-FRP ☆

Chaitanya Belwal *, Albert M.K. Cheng, Bo Liu

Department of Computer Science, University of Houston, TX, USA

ARTICLE INFO

Article history:
Received 19 February 2012
Received in revised form 7 October 2012
Accepted 8 November 2012
Available online 13 December 2012

Keywords:
Real-time systems
Schedulability
Functional programming
Feasibility interval
Response time analysis

ABSTRACT

Functional Reactive Programming (FRP) is a resource aware declarative approach for modeling and building safety-critical embedded systems. Recently, Priority-based FRP (P-FRP) was introduced as a formalism that guarantees real-time response. Due to the state-less nature of execution of functional programs, P-FRP implements a transactional nature of execution where preempted lower-priority tasks are aborted. This makes the response time of a lower-priority task completely dependent on the execution pattern of higher priority tasks. The feasibility interval in the classical preemptive model[1] of real-time systems is known and is dependent on the least common multiple (LCM) of task periods. However, since the abort nature of preemption can induce side-effects on the execution of lower-priority tasks, it has been unknown to date if the feasibility in P-FRP is also dependent on the LCM. In this paper, we rigorously prove that these side-effects of preemption are bounded within the LCM and formally derive a value of the feasibility interval in P-FRP. This value of feasibility interval is vital for more robust schedulability analysis of the P-FRP execution model.

© 2012 Elsevier Inc. All rights reserved.

1. Introduction

Functional Reactive Programming (FRP) [27] is a declarative programming language for the modeling and implementation of safety-critical embedded systems. It has been used for a wide range of applications, notably, graphics [13], robotics [22], and vision [23]. FRP elegantly captures the continuous and discrete aspects of a hybrid system using the notions of behavior and event, respectively. Because this language is developed as an embedded language in Haskell [16], it benefits from the wealth of abstractions provided in this language. Unfortunately, Haskell provides no real-time guarantees, and therefore, neither does FRP.

To address this limitation, resource-bounded variants of FRP were studied [25–27]. Recently, it was shown that a variant called priority-based FRP (P-FRP) [17] combines both the semantic properties for FRP, guarantees resource boundedness, and supports assigning different priorities to different events. In P-FRP, higher priority events can preempt lower-priority ones. However, to maintain guarantees of type-safety and state-less execution the functional programming paradigm requires the execution of a function to continue uninterrupted. To comply with this requirement and allow preemption of lower-priority events, P-FRP implements a transactional model of execution. Using only a copy of the state during event execution and atomically committing these changes at the end of the event handler, P-FRP ensures that handling an event is an "all or

☆ This work is supported in part by US National Science Foundation under award Nos. 0720856 and 1219082.

* Corresponding author.
E-mail addresses: cbelwal@cs.uh.edu (C. Belwal), cheng@cs.uh.edu (A.M.K. Cheng), boliu@cs.uh.edu (B. Liu).

[1] In this paper the classical preemptive model refers to a real-time system in which tasks can be preempted by higher priority tasks and can resume execution from the point they were preempted.

0022-0000/$ – see front matter © 2012 Elsevier Inc. All rights reserved.
http://dx.doi.org/10.1016/j.jcss.2012.11.006

nothing" proposition. This preserves the easily understandable semantics of FRP and provides a programming model where response times to different events can be tweaked by the programmer without ever affecting the semantic soundness of the program. Thus, a clear separation between the semantics of the program and the responsiveness of the implementation of each event handler (or task) is achieved.

A real-time system can run for an unbounded amount of time, and no task should miss its deadline as long as the system is running (assuming all system parameters remain the same). Since it is not feasible to ascertain the schedulability for an unbounded period of time, a finite value to study schedulability is required. This finite amount of time for which the schedulability has to be analyzed, is termed as the feasibility interval in real-time studies.

In their seminal paper, Liu and Layland [20] have shown that the *Worst-Case Response Time* (*WCRT*) for any task in the preemptive model occurs when it is released *synchronously* (at the same time) as higher priority tasks. This is also known as the *critical instant* of release. For the synchronous release of tasks under the preemptive model, a common feasibility interval used is $[0, L)$, where L represents the *least common multiple* (LCM) of all the task periods in the system. In the preemptive model, if the task set is schedulable in the period $[0, L)$ it is guaranteed to be always schedulable since the synchronous release is the worst-case release scenario.

In [4], it has been shown that unlike the preemptive model in P-FRP, a synchronous release of tasks does not lead to the WCRT for all cases. In [4,8] a method to compute the response time of a P-FRP under a given release scenario has been given. This method is used to compute the WCRT of a P-FRP task, by analyzing the response time under all possible release scenarios of higher priority tasks. The highest response time computed in this way is the WCRT of the task. Hence, in P-FRP it is possible for an *asynchronous* (at different times) release of tasks to lead to WCRT, therefore derivation of feasible interval under such a release scenario is required. However due to different nature of execution in which preempted tasks are aborted, it is unknown if any of the results for the preemptive model can be applied to this execution model. Therefore a fresh approach in determining the feasibility interval in P-FRP is required, which has been provided in this paper.

1.1. Related work

Important work on feasibility interval for asynchronous preemptive systems has been done by several authors [3,14, 18,19]. A detailed study was done by Leung and Whitehead [18] and extended by Goossens and Devillers [14], who give detailed proof for deriving the feasibility interval under various scenarios. Audsley [2] has used Leung and Whitehead's feasibility interval for determining schedulability and optimal priority assignment with asynchronous release of tasks. However, these previous works determine the feasibility interval only for the preemptive execution model and for situations where the critical instant of task releases is known. In the P-FRP execution model, preempted tasks have to restart leading to a completely different execution semantics than the preemptive model. As shown in [4], due to the restart of preempted tasks, a single critical instant of release which leads to the WCRT of a lower-priority task does not exist in P-FRP. With these fundamental differences in the preemptive and P-FRP execution models, it is unknown if the feasibility interval derived for the preemptive model holds true for the transactional execution model of P-FRP. Hence, the results of these previous works cannot be applied to determine the feasibility interval of P-FRP task sets and a new approach is required.

Several studies on response time and schedulability analysis of P-FRP are available. In [24], a method based on Audsley's iterative approach [1] has been presented to derive the WCRT of a task while in [17] formal schedulability conditions for P-FRP have been presented. In [4,5,8] methods to compute actual response time of P-FRP under a known release scenario have been presented. In [6], the efficiency of three first-fit heuristics for static partitioning of P-FRP tasks in multi-processor platforms has been analyzed, while a utilization based sufficient schedulability test for P-FRP is derived in [7]. In [10] an efficient approach to compute the exact worst-case response time using an optimized search space is presented, and possibility of an optimal priority assignment for P-FRP tasks has been briefly presented in [11]. None of these prior works on P-FRP deal with the feasibility interval of P-FRP task sets. Although a feasibility interval is provided in [12], it only addresses the fixed-priority scheduler on uniform multiprocessor platform.

Apart from studies on P-FRP, several authors have presented the real-time properties of similar transaction-oriented execution models like lock-free [1], software transaction memory [15] and atomic critical section in Java [21]. These studies however, derive basic schedulability conditions or provide response time analysis without consideration of the feasibility interval. So far, the feasibility interval for P-FRP and similar transaction based execution models has not been presented.

1.2. Contributions

In this paper, we present the execution semantics of P-FRP and derive a feasibility interval for its execution model. After presenting the basic concepts and assumptions made in this study (Section 2), we present a necessary scheduling condition that characterizes an important relationship between periods and processing time of P-FRP tasks (Section 3). We then formally present the execution characteristics of tasks in a P-FRP system with 2 tasks (Section 4). The execution characteristics of this system is then applied to a general P-FRP task set with n tasks after which the feasibility interval in P-FRP is derived (Section 4). We provide details of the experiments used for validating the feasibility interval (Section 5), and finally conclude with a reflection on our results (Section 6).

2. Basic concepts and assumptions

In this section, we introduce the basic concepts and the notation used in this paper. In addition, we review the P-FRP execution model and assumptions made in this study. For formal description of the P-FRP language semantics, readers are advised to refer the original P-FRP paper [17].

2.1. Basic concepts

Essential concepts for P-FRP are tasks and their associated priority, their associated time period and the concept of arrival rate and their processing time; the concept of a time interval and task jobs therein. The notation and formal definitions for these concepts as well as a few others used in the paper are as follows:

- Let task set $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$ be a set of n periodic tasks. Tasks can represent processes or threads.
- Each unique invocation of a task is called its **job**.
- $\tau_i(k)$ represents the job of τ_i which is released at time k.
- The **priority** of a task $\tau_k \in \Gamma_n$ is the positive integer k, where a higher number implies higher priority. Examples of higher priority tasks are critical interrupts while low priority tasks can be normal applications.
- T_k is the **time period** between two successive jobs of τ_k.
- C_k is the **worst-case execution time** (**WCET**) for τ_k.
- $t_{copy}(k)$ is the time taken to make a **copy** of the state before τ_k starts execution.
- $t_{restore}(k)$ is the time taken to **restore** the state after τ_k has completed execution (details provided in the following section).
- P_k is the **processing time** for τ_k. Processing of a task includes execution as well as copy and restore operations. Hence, $P_k = t_{copy}(k) + C_k + t_{restore}(k)$.
- An **absolute time** t (or **time** t) is the time elapsed since the real-time system is started. The real-time system is assumed to have started at absolute time 0.
- $[t_1, t_2)$ represents a time interval such that: $\forall t \in [t_1, t_2)$, $t_1 \leqslant t < t_2 \wedge t_1 \neq t_2$. t_1 and t_2 are absolute times.
- $R_{k,m}$ represents the **release time** of the mth job of τ_k.
- Φ_k represents the **release offset**, which is the release time of the first job of τ_k. Or, $\Phi_k = R_{k,1}$. Hence, $R_{k,m} = \Phi_k + (m-1) \cdot T_k$.
- $[t_1, t_2)$ is a ***k*-idle period** if for every $t \in [t_1, t_2)$ there is no task having priority k or higher awaiting execution and ready to execute strictly before t.
- D_k is the **relative deadline** of τ_k. After its release τ_k should complete processing within D_k time units, otherwise it will have a **deadline miss**. For this study we assume $D_k = T_k$.
- Two time intervals of equal lengths $[t_1, t_1 + a)$ and $[t_2, t_2 + a)$ are said to have the same **processing pattern**, if for every value of relative time t: $0 \leqslant t < a$, the task that is processed at relative time t in $[t_1, t_1 + a)$ (absolute time $t_1 + t$), is also processed at relative time t in $[t_2, t_2 + a)$ (absolute time $t_2 + t$).
- A **feasibility interval** is the time interval $[t, t + H)$ such that if all tasks are schedulable in $[t, t + H)$ then the tasks will also be schedulable in the time interval $[0, Z)$: $Z \to \infty$.
- **Interference** on τ_k is the action where the processing of τ_k is interrupted by the release of a higher priority task.

2.2. Execution model and assumptions

In this study, all tasks are assumed to run in a uniprocessor system. The task processing times, arrival periods and release times of the first jobs are assumed to be known *a priori*. When a job of a higher priority task is released, it can immediately preempt a lower-priority task and changes made by the lower-priority task are rolled back. The lower-priority task will be restarted after the higher priority task has completed processing. When some task is released, it enters a processing queue which is arranged by priority order such that all arriving jobs of higher priority tasks are moved to the head of the queue. The length of the queue is bounded and no two jobs of the same task can be present in the queue at the same time. This requires a task to complete processing before the release of its next job. To maintain this requirement, we assume a hard real-time system with task processing deadline equal to the minimal necessary wait. Once a task τ_i enters the queue, two situations are possible. If a task of lower priority than i is being processed, it will be immediately preempted and τ_i will start processing. If a task of higher priority than i is being processed, then τ_i will wait in the queue and start processing only after the higher priority task being processed, has completed. An exception to the immediate preemption is made during copy and restore operations which is explained in the following paragraph.

2.3. Copy and restore operation

In P-FRP, when a task starts processing it creates a 'scratch' state which is a copy of the current state of the system. Changes made during the processing of this task are maintained inside such a state. When the task has completed, the 'scratch' state is restored into the final state in an atomic operation. Therefore, the restoration and copy operations are

guarded such that the task being processed cannot be preempted by higher priority tasks. If the task is preempted after copy but before the restore operation, the scratch state is simply discarded. The time to discard the state of an aborted task is minimal and has been ignored in this study. The context-switch between tasks only involves a state copy operation for the task that will be commencing processing. The time taken for copy (t_{copy}) and restore ($t_{restore}$) operations of some task τ_k is part of its processing time, P_k.

3. A necessary scheduling condition

In this section, we present a condition that has to be satisfied by every schedulable task set. This condition is required in the proofs used for deriving the feasibility interval.

Lemma 1. *Consider $\Gamma_2 = \{\tau_1, \tau_2\}$. For Γ_2 to be schedulable, the following conditions will always be satisfied*:

$$T_1 \geqslant P_1 + P_2 \quad \textit{and} \quad T_2 \geqslant P_1 + P_2.$$

Proof. There are two possible conditions based on the relationship between T_1 and T_2.

Case 1. $T_2 < T_1$.

If both tasks are released simultaneously then τ_2 will complete first. τ_1 will start processing when τ_2 has completed which is at time $0 + P_2$. The next job of τ_2 will be released at time T_2. The time left for processing τ_1 is $T_2 - P_2$. If τ_1 is unable to complete processing within this time, it will be aborted by the 2nd job of τ_2 which is released at time T_2. After the 2nd job of τ_2 has completed, τ_1 will get another time period of length $T_2 - P_2$ to complete processing. The abort/restart cycle of τ_1 will continue for every job of τ_2. If $P_1 > T_2 - P_2$, τ_1 will never be able to complete processing and the task set will be unschedulable. Hence, if Γ_2 is schedulable τ_1 will be able to complete processing between successive jobs of τ_2. Or,

$$P_1 \leqslant T_2 - P_2. \tag{1.1}$$

Or,

$$P_2 \leqslant T_2 - P_1.$$

Since $T_2 < T_1$,

$$P_2 \leqslant T_1 - P_1. \tag{1.2}$$

Case 2. $T_2 \geqslant T_1$.

If both tasks are released simultaneously, then τ_2 will complete first. τ_1 will start processing when τ_2 has completed which is at time $0 + P_2$. The next job of τ_1 will be released at time T_1, before the 2nd job of τ_2 is released. The time left for processing τ_1 is $T_1 - P_2$. If τ_1 is unable to complete processing within this time it will miss its deadline. Hence if Γ_2 is schedulable, then τ_1 will be able to complete processing between its two successive jobs after accounting for the processing of τ_2. Therefore,

$$P_1 < T_1 - P_2. \tag{1.3}$$

Or,

$$P_2 < T_1 - P_1. \tag{1.4}$$

In Eq. (1.3), since $T_2 \geqslant T_1$

$$P_1 < T_2 - P_2. \tag{1.5}$$

Eqs. (1.1), (1.2), (1.4) and (1.5) show that if a task set Γ_2 is schedulable, each task will be able to complete processing between jobs of the other task present in the set. □

4. Deriving the feasibility interval

We now formally present the derivation of the feasibility interval in P-FRP. We show that a task will be released at the same relative times in two consecutive intervals of length L (L represents the LCM of periods of all tasks present in Γ_n), and, subsequently that two consecutive intervals have the same processing pattern. We then derive the theorem for feasibility interval for both asynchronous and synchronous releases of tasks.

Property 1. *Consider $\Gamma_2 = \{\tau_1, \tau_2\}$. If some job of τ_2 is processed in the time interval $[t, t + P_2)$, then another job of τ_2 will be processed in the time interval $[t + L, t + P_2 + L)$.*

Proof. Let a job of τ_2 be released at time t (represented by $\tau_2(t)$), hence another job of τ_2 be released at time $t+L$ (represented by $\tau_2(t+L)$). Since τ_2 has the higher priority, it cannot be preempted by any other task. Hence, $\tau_2(t)$ will complete processing at time $t+P_2$, while $\tau_2(t+L)$ will complete at time $t+P_2+L$. Hence, $\tau_2(t)$ is processed in the time interval $[t, t+P_2)$ while $\tau_2(t+L)$ is processed in the time interval $[t+L, t+P_2+L)$. □

Lemma 2(a). *Consider $\Gamma_2 = \{\tau_1, \tau_2\}$. If some job of τ_1 first starts processing at some time t_{start}, then another job of τ_1 will also first start processing at time $t_{start}+L$.*

Proof. Let a job of τ_1 be released at time t (represented by $\tau_1(t)$), hence another job of τ_1 be released at time $t+L$ (represented by $\tau_1(t+L)$). Let there be a job of τ_2 released at time t_x such that it causes interference or delay in the processing of $\tau_1(t)$, hence a job of τ_2 will also be released at time t_x+L. There are three possible cases based on the value of t_x:

Case 1. $t_x < t$.

Since $\tau_2(t_x)$ is released before $\tau_1(t)$, $\tau_2(t_x)$ can only delay the processing of $\tau_1(t)$. To cause a delay, $\tau_2(t_x)$ should not complete before time t. Or, the following condition should be satisfied:

$$t_x > t - P_2.$$

$\tau_2(t_x)$ will complete at time t_x+P_2, after which $\tau_1(t)$ will start processing. Hence $t_{start} = t_x + P_2$. Similarly, $\tau_2(t_x+L)$ will delay the processing of $\tau_2(t+L)$. $\tau_2(t_x+L)$ will complete processing at time t_x+L+P_2, after which $\tau_1(t+L)$ will start at time:

$$t_x + L + P_2.$$

Or, $\tau_1(t+L)$ will only start processing at time $t_{start}+L$.

Case 2. $t_x = t$.

$\tau_2(t_x)$ will be released at the same time as $\tau_1(t)$. Since, it has a higher priority, $\tau_2(t_x)$ will start processing first and complete at time t_x+P_2, after which $\tau_1(t)$ will start processing. Hence, $t_{start} = t_x + P_2$.

$\tau_2(t_x+L)$ and $\tau_1(t+L)$ will also be released at the same time. $\tau_1(t+L)$ will start processing only after $\tau_2(t_x+L)$ has completed at time t_x+P_2+L. Or, $\tau_1(t+L)$ will start processing at time $t_{start}+L$.

Case 3. $t_x > t$.

$\tau_2(t_x)$ will be released after $\tau_1(t)$ has started processing. Hence, the first processing of $\tau_1(t)$ will start as soon as it is released, or $t_{start} = t$. Since, $\tau_2(t_x+L)$ will be released after $\tau_1(t+L)$, $\tau_1(t+L)$ will also start processing as soon as it is released. Or, $\tau_1(t+L)$ will start processing at times $t_{start}+L$. □

Lemma 2(b). *Consider $\Gamma_2 = \{\tau_1, \tau_2\}$. If some job of τ_1 completes processing at time $t_{complete}$, then another job of τ_1 will also complete processing at time $t_{complete}+L$.*

Proof. Let a job of τ_1 be released at time t (represented by $\tau_1(t)$), hence another job of τ_1 will be released at time $t+L$ (represented by $\tau_1(t+L)$). Let there be a job of τ_2 released at time t_x such that it causes interference or delay in the processing of $\tau_1(t)$, hence a job of τ_2 will be released at time t_x+L. There are five possible cases based on the value of t_x:

Case 1. $t - P_2 < t_x < t$.

$\tau_2(t_x)$ is released before $\tau_1(t)$, hence $\tau_1(t)$ will not be able to start processing till $\tau_2(t_x)$ has completed. $\tau_2(t_x)$ will complete processing at t_x+P_2, after which $\tau_1(t)$ will start and will complete processing at time $t_x+P_2+P_1$. Hence, $t_{complete} = t_x + P_2 + P_1$.

$\tau_2(t_x+L)$ is also released before $\tau_1(t+L)$, and $\tau_1(t+L)$ will start processing after $\tau_2(t_x+L)$ has completed processing at time t_x+P_2+L. Or, $\tau_1(t+L)$ will complete processing at time $t_x+P_2+P_1+L$, which is the same as $t_{complete}+L$.

Case 2. $t_x = t$.

$\tau_2(t_x)$ will be released at the same time as $\tau_1(t)$. Since, it has a higher priority $\tau_2(t_x)$ will start processing first and complete at time t_x+P_2, after which $\tau_1(t)$ will start processing and complete at time $t_{complete} = t_x + P_2 + P_1$.

$\tau_2(t_x+L)$ and $\tau_1(t+L)$ will also be released at the same time. $\tau_1(t+L)$ will start processing only after $\tau_2(t_x+L)$ has completed at time $t_x+P_2+P_1+L$, which is the same as $t_{complete}+L$.

Case 3. $t < t_x < t + t_{copy}(1)$.

$\tau_2(t_x)$ will be released while $\tau_1(t)$ is doing the copy operation. Hence, $\tau_2(t_x)$ will not be able to abort $\tau_1(t)$ immediately, but will abort it as soon as the copy operation is complete at time $t + t_{copy}(1)$. After $\tau_2(t_x)$ completes at time $t + t_{copy}(1) + P_2$, $\tau_1(t)$ will restart and complete at time $t_{complete} = t + t_{copy}(1) + P_2 + P_1$.

$\tau_2(t_x + L)$ will also abort $\tau_1(t + L)$ at time $t + t_{copy}(1) + L$, and will complete at time $t + t_{copy}(1) + L + P_2$. $\tau_1(t + L)$ will restart at time $t + t_{copy}(k) + L + P_2$ and complete by time $t + t_{copy}(1) + L + P_2 + P_1$ which is the same as $t_{complete} + L$.

Case 4. $t + t_{copy}(1) \leqslant t_x < t + t_{copy}(1) + C_1$.

$\tau_2(t_x)$ will abort $\tau_1(t)$ and start processing as soon as it is released at time t_x. In this case, an abort of $\tau_1(t)$ by $\tau_2(t_x)$ is guaranteed. $\tau_2(t_x)$ will complete at time $t_x + P_2$, after which $\tau_1(t)$ will restart processing and complete at time $t_{complete} = t_x + P_2 + P_1$.

$\tau_2(t_x + L)$ will also abort $\tau_1(t + L)$ and start processing as soon as it is released at time $t_x + L$. $\tau_2(t_x + L)$ will complete at time $t_x + L + P_2$, after which $\tau_1(t)$ will restart processing and complete at time $t_x + L + P_2 + P_1$, which is the same as $t_{complete} + L$.

Case 5. $t + t_{copy}(1) + C_1 < t_x < t + t_{copy}(1) + C_1 + t_{restore}(1)$.

$\tau_2(t_x)$ will be released when $\tau_1(t)$ is doing the restore operation. Hence, $\tau_2(t_x)$ will not be able to abort $\tau_1(t)$. At time $t + t_{copy}(1) + C_1 + t_{restore}(1)$, $\tau_1(t)$ will complete processing, allowing $\tau_2(t_x)$ to start. In this case, $t_{complete} = t + t_{copy}(1) + C_1 + t_{restore}(1) = t + P_1$.

$\tau_2(t_x + L)$ will also be released when $\tau_1(t + L)$ is doing the restore operation. Hence, $\tau_2(t_x + L)$ will not be able to abort $\tau_1(t + L)$. At time $t + L + t_{copy}(1) + C_1 + t_{restore}(1)$, $\tau_1(t + L)$ will complete processing, allowing $\tau_2(t_x)$ to start. Or, $\tau_1(t + L)$ will complete processing at time $t_{complete} + L$.

Case 6. $t_x > t_{copy}(1) + C_k + t_{restore}(1)$.

In this case, $\tau_2(t_x)$ will be released after $\tau_1(t)$ has completed processing. Since, $\tau_1(t)$ not be preempted in this case, $t_{complete} = t + P_1$.

$\tau_2(t_x + L)$ will also be released after $\tau_1(t + L)$ has completed processing. $\tau_1(t + L)$ will complete at time $t + L + P_1 = t_{complete} + L$.

Note in all these cases we have accounted for only one job of τ_2 that can abort or cause delay in the processing τ_1. This is because if the task set is schedulable it will satisfy the necessary schedulability test (Lemma 1), according to which only one job of τ_2 can be released between the release and completion of τ_1. □

Lemma 3. *Consider $\Gamma_2 = \{\tau_1, \tau_2\}$ where the first jobs of τ_1 and τ_2 are released at times Φ_1 and Φ_2. Let R_{max} denote the maximum release time of these tasks. Or $R_{max} = \max\{\Phi_1, \Phi_2\}$. Then the processing pattern in the interval $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$ will be the same.*

Proof. Consider two linear tables with L blocks. Each block in the table represents a single unit of discrete time, which can be marked with a task identifier. Table 1 represents the time interval $[R_{max}, R_{max} + L)$, while table 2 represents $[R_{max} + L, R_{max} + 2 \cdot L)$. The processing of tasks in these time intervals can be represented by their respective tables. There are three possible cases depending on the values of Φ_1 and Φ_2, and an analysis of these is given below. Examples that illustrate these cases are also provided.

Case 1. $R_{max} = \Phi_2$.

From Property 1, we know that for each job of τ_2 which starts processing at time t, there will be another job that will also start its processing at time $t + L$. The first job of τ_2 will start processing at time R_{max}, hence the starting blocks (index 1) in both tables are marked with τ_2. The first job will complete processing at times $R_{max} + P_2$ and $R_{max} + L + P_2$ respectively. We mark all the blocks from index 1 to P_2 in both the tables. This process is repeated for every job of τ_2 in the interval $[R_{max}, R_{max} + L)$.

Next we start analyzing jobs of τ_1. From Lemmas 2(a) and 2(b) we know if a job of τ_1 starts processing at time t_{start} and completes at time $t_{complete}$, a job of τ_1 will also start at time $t_{start} + L$ and complete at time $t_{complete} + L$. Hence, all unmarked blocks between t_{start} and $t_{complete}$ should be marked in table 1 with τ_1, while the same blocks should be marked in table 2 (to represent processing of the job of τ_1 which starts and completes at times $t_{start} + L$ and $t_{complete} + L$, respectively). This process is repeated for every job of τ_1 in the interval $[R_{max}, R_{max} + L)$.

Note that if some job of τ_1 is aborted by the release of τ_2 at time R_{max}, then the aborted job will complete at time $R_{max} + P_2 + P_1$. Per Lemma 2(b), there will be another job of τ_1 that will complete at time $R_{max} + P_2 + P_1 + L$. If there is

no job of τ_1 executing at time R_{max}, then this job will not be considered in the intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$.

Case 2. $R_{max} = \Phi_1$.

If a job of τ_2 is being processed at time R_{max}, it will continue processing till it completes at time t ($t > R_{max}$). Per Property 1 another job of τ_2 will complete at time $t+L$, hence the first $R_{max} - t$ blocks on the two tables are marked with τ_2. Others blocks are also marked to account for the processing of jobs of τ_2 in the interval $[R_{max}, R_{max} + L)$. After processing of τ_2 has been accounted for, blocks in the two tables are marked to account for all jobs of τ_1 based on Lemmas 2(a) and 2(b).

If no job of τ_2 is being processed at time R_{max}, then the initial blocks that represent the times at which τ_2 will not be processed are kept unmarked. Other blocks in the two tables that denote the processing of all jobs of τ_2 in the intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$ are still marked first. Then the blocks are marked based on the processing of all jobs of τ_1, with the first job starting at time R_{max}.

Case 3. $R_{max} = \Phi_1 = \Phi_2$.

In this case, τ_2 will execute first followed immediately by τ_1. We first mark blocks in the two tables to represent the processing of jobs of τ_2 based on Property 1. We then mark all the blocks to account for the processing of τ_1 based on Lemmas 2(a) and 2(b).

For all the three cases, when blocks are marked this way each corresponding block in the two tables will have the same markings. This denotes that the processing pattern in intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$ is the same. □

Example. Consider the following 2-task set:

Task	*T*	*P*
τ_1	9	3
τ_2	6	3

For this example, the time for copy and restore operation is considered 1. Or, $t_{copy}(1) = t_{copy}(2) = 1$ and $t_{restore}(1) = t_{restore}(2) = 1$. The LCM of τ_1 and τ_2 is 18. Hence $L = 18$. Using this task set we will analyze all these 3 cases, based on different sample values of Φ_1 and Φ_2.

Case 1. $\Phi_1 = 0$, $\Phi_2 = 1$.

In this case, $R_{max} = 1$. First we mark blocks to account for the processing of all jobs of τ_2 in the intervals $[1, 19)$ (table 1) and $[19, 37)$ (table 2) based on Property 1. There are 3 jobs of τ_2 and the tables are represented in Figs. 1(a) and 1(b) respectively. Next, the tables are marked to account for the processing of 3 jobs of τ_1 in intervals $[1, 19)$ and $[19, 37)$ based on Lemmas 2(a) and 2(b), and the results are shown in Figs. 1(c) and 1(d) respectively. The processing patterns in the two tables representing the two intervals $[1, 19)$ and $[19, 37)$ match. Note that the 1st of τ_1 which starts at time 0, is aborted by the release of the 1st job of τ_2 at time 1. Also the start time of the 2nd job of τ_1 is delayed by the processing of the 2nd job of τ_2.

Case 2. $\Phi_1 = 2$, $\Phi_2 = 1$.

In this case, $R_{max} = 2$. First we mark blocks to account for the processing of all jobs of τ_2 in the intervals $[2, 20)$ (table 1) and $[20, 38)$ (table 2) based on Property 1. There are 3 jobs of τ_2 and the tables are represented in Figs. 1(e) and 1(f) respectively. Next, the tables are marked to account for the 2 jobs of τ_1 in intervals $[2, 20)$ and $[20, 38)$, based on Lemmas 2(a) and 2(b) and the results are shown in Figs. 1(g) and 1(h) respectively. The processing patterns in the two tables representing the two intervals match. In this case, the start of the 1st job of τ_1 is delayed by the processing of the 1st job of τ_2. Also the 2nd job of τ_1 is aborted by the release of the 3rd job of τ_2.

Case 3. $\Phi_1 = 0$, $\Phi_2 = 0$.

In this case, $R_{max} = 0$. First we mark blocks to account for the processing of all jobs of τ_2 in the intervals $[0, 18)$ (table 1) and $[18, 36)$ (table 2) based on Property 1. There are 3 jobs of τ_2 and the tables are represented in Figs. 1(i) and 1(j) respectively. Next, the tables are marked to account for the 2 jobs of τ_1 in the intervals $[0, 18)$ and $[18, 36)$, based on Lemmas 2(a) and 2(b) and the results are shown in Figs. 1(k) and 1(l) respectively. The processing patterns in the two tables representing the two intervals match.

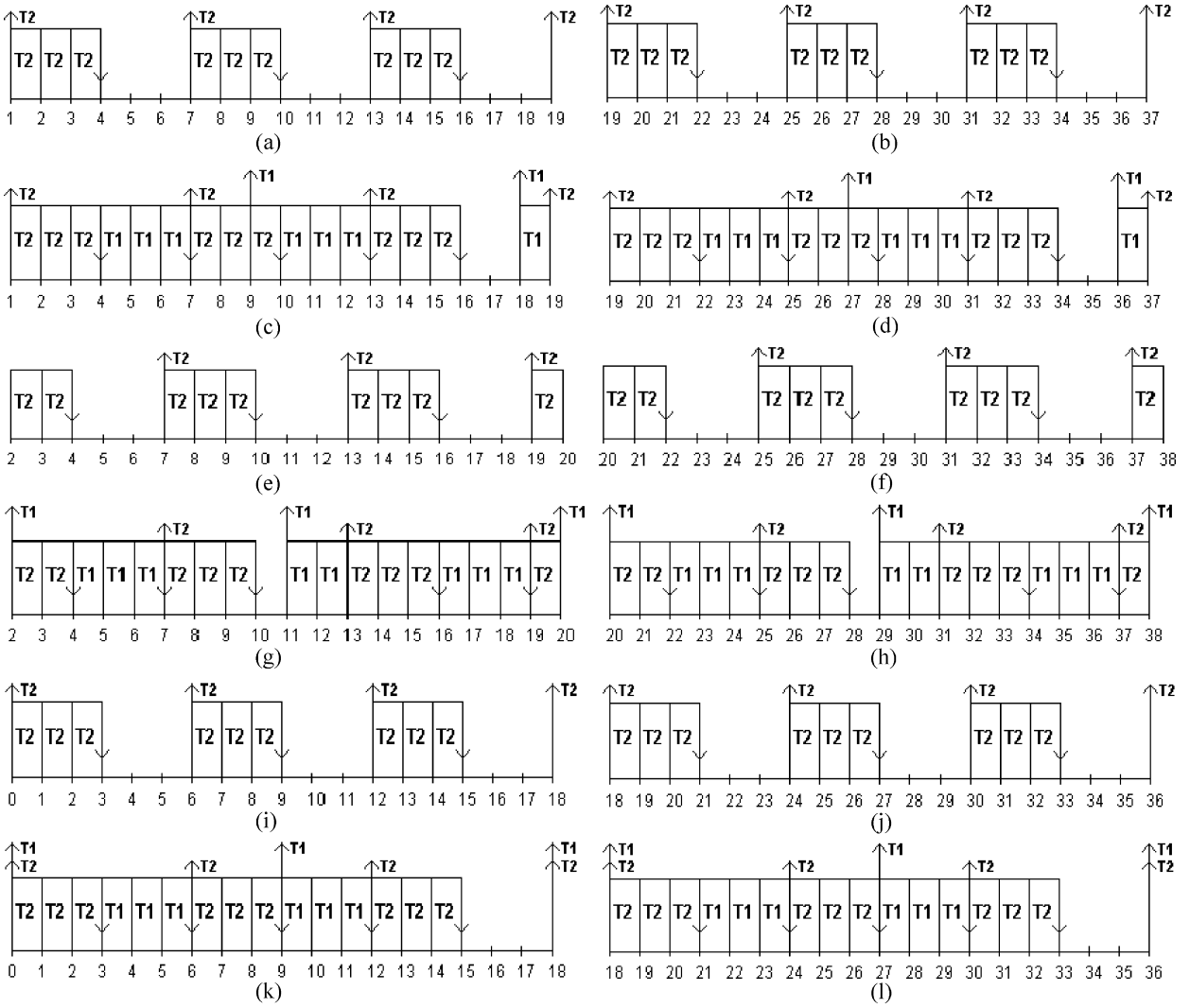


Fig. 1. Graphical representation of tables with marked blocks showing processing of tasks τ_1 and τ_2 when they are released at different times. T1, T2 represent tasks τ_1 and τ_2 respectively. For (a), (b), (c) and (d) the release times of τ_1, τ_2 are 0, 1 respectively. For (e), (f), (g) and (h) release times are 2, 1 while for (i), (j), (k) and (l) they are 0, 0. (a) Processing of τ_2 in the interval [1, 19). (b) Processing of τ_2 in the interval [19, 37). (c) Processing of τ_1, τ_2 in the interval [1, 19). (d) Processing of τ_1, τ_2 in the interval [19, 37). (e) Processing of τ_2 in the interval [2, 20). (f) Processing of τ_2 in the interval [20, 38). (g) Processing of τ_1, τ_2 in the interval [2, 20). (h) Processing of τ_1, τ_2 in the interval [20, 38). (i) Processing of τ_2 in the interval [0, 18). (j) Processing of τ_2 in the interval [18, 36). (k) Processing of τ_1, τ_2 in the interval [0, 18). (l) Processing of τ_1, τ_2 in the interval [18, 36).

Lemma 4. *Consider $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$ where the first job of $\tau_i \in \Gamma_n$ is released at time Φ_i. Let R_{max} denote the maximum release time of these tasks. Or $R_{max} = \max_{i=1}^{n}\{\Phi_i\}$. Then the processing pattern in the interval $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$ will be the same.*

Proof. From Lemma 3, we know that if $n = 2$ and tasks are released at times Φ_1 and Φ_2, the processing patterns in the intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$, where $R_{max} = \max\{\Phi_1, \Phi_2\}$ remain the same. The method used to prove Lemma 3 can be extended to a general task set $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$. Assume two linear tables with L blocks. Each block in the table represents a single unit of discrete time, and table 1 represents the time interval $[R_{max}, R_{max} + L)$, while table 2 represents $[R_{max} + L, R_{max} + 2 \cdot L)$, where $R_{max} = \max_{i=1}^{n}\{\Phi_i\}$.

Consider the set $\Gamma_2 = \{\tau_{n-1}, \tau_n\}$: $\Gamma_2 \subset \Gamma_n$. From Property 1, we know that each job of τ_n that starts processing at time t will also start processing at time $t + L$. Since, $\Phi_n, \Phi_{n-1} \leqslant R_{max}$, jobs of τ_{n-1}, τ_n can be marked in both the tables. If some job of τ_n starts processing at time t: $R_{max} < t \leqslant R_{max} + L$, another job of τ_n will start processing at time $t + L$. These jobs will complete processing at times $t + P_n$ and $t + L + P_n$ respectively. We mark all the blocks that represent the time interval $[t, t + P_n)$ in table 1 and the interval $[t + L, t + P_n)$ in table 2. This process is repeated for every job of τ_n in the interval $[R_{max}, R_{max} + L)$. Next we start analyzing jobs of τ_{n-1}, and mark the blocks in the two tables based on Lemmas 2(a)

and 2(b) which state that if job of τ_{n-1} starts processing at time t_{start} and completes at time $t_{complete}$, a job of τ_{n-1} will also start at time $t_{start} + L$ and complete at time $t_{complete} + L$.

Next consider the set $\Gamma_2 = \{\tau_{n-2}, \tau_{n-1}\}$: $\Gamma_2 \subset \Gamma_n$. Again, since, $\Phi_{n-1}, \Phi_{n-2} \leqslant R_{max}$, jobs of τ_{n-2}, τ_{n-1} can be marked in both the tables. The lower-priority task τ_{n-2} can only be processed in the $(n-1)$-idle-periods remaining after accounting for the processing of $\{\tau_{n-1}, \tau_n\}$ in the intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$. Since, the processing pattern in these two intervals, after accounting for the processing of τ_n and τ_{n-1} will be the same per Lemma 3, the same number and size of $(n-1)$-idle-periods will be present in these two intervals. It is easy to see that Lemmas 2(a) and 2(b) can also be applied to the task τ_{n-2}. Hence, for every job of τ_{n-2} that starts and completes by times t_{start} and $t_{complete}$ respectively, there will be a job of τ_{n-2} that starts and completes at times $t_{start} + L$ and $t_{complete} + L$. When blocks to denote the processing of all jobs of τ_{n-2} are marked, we consider the task set $\{\tau_{n-3}, \tau_{n-2}\}$: $\Gamma_2 \subset \Gamma_n$ and mark blocks to denote the processing of all jobs of τ_{n-3}.

This way we continue marking blocks till processing of all jobs of τ_1 have been accounted for. The contents of each corresponding block in both the tables will be the same, denoting that the processing pattern in intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$ is the same. □

Example. We illustrate this lemma using a task set with 3 tasks. The details of this task set are given below:

Task	T	C
τ_1	18	2
τ_2	12	4
τ_3	9	3

Since the LCM of T_1, T_2 and T_3 is 36, $L = 36$. For the sake of brevity, we illustrate only one release scenario where all tasks are released synchronously at time 0. Hence, $R_{max} = 0$. First we mark blocks to account for the processing of all jobs of τ_3 in the interval $[0, 36)$ (table 1), and $[36, 72)$ (table 2) based on Property 1. There are 4 jobs of τ_3 and the tables are represented in Figs. 2(a) and 2(b) respectively. Next, the tables are marked to account for the 3 jobs of τ_2 in each interval $[0, 36)$ and $[36, 72)$, with the resulting tables shown in Figs. 2(c) and 2(d) respectively. Finally, we mark the blocks to account for the processing of 2 jobs of τ_1 and the tables are shown in Figs. 2(e) and 2(f). The processing pattern in the two tables representing the two intervals $[0, 36)$ and $[36, 72)$ match.

Lemma 5. *Consider $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$ and $R_{max} = \max_{i=1}^{n}\{\Phi_i\}$. Then the processing pattern in the two consecutive intervals $[R_{max} + m \cdot L, R_{max} + (m+1) \cdot L)$ and $[R_{max} + (m+1) \cdot L, R_{max} + (m+2) \cdot L)$, $m \geqslant 0$, will be the same.*

Proof. When $m = 0$, the processing pattern in the intervals $[R_{max}, R_{max} + L)$ and $[R_{max} + L, R_{max} + 2 \cdot L)$, are the same per Lemma 4. Let τ_j denote the task where $R_{max} = \Phi_j$. Jobs of τ_j are guaranteed to be released at times R_{max}, $R_{max} + L$, $R_{max} + 2 \cdot L$ and so on. Or, jobs of τ_j are guaranteed to be released at time $R_{max} + m \cdot L$: $m \geqslant 0$. By setting, $R_{max} = R_{max} + m \cdot L$: $m > 0$, and using Lemma 4 it is easy to see that the processing pattern in the two consecutive intervals $[R_{max} + m \cdot L, R_{max} + (m+1) \cdot L)$ and $[R_{max} + (m+1) \cdot L, R_{max} + (m+2) \cdot L)$: $m \geqslant 0$ will be the same. □

Example. Consider the task set used in Lemma 4. In Figs. 3(a) and 3(b) we show the processing of tasks in intervals $[72, 108)$ and $[108, 144)$. Comparing with Figs. 2(e) and 2(f), it is easily discerned that the processing pattern in all the four intervals when $m = 0$: $[0, 36)$, $m = 1$: $[36, 72)$, $m = 2$: $[74, 108)$ and $m = 3$: $[108, 144)$ is the same.

Lemma 6. *Consider $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$ and $R_{max} = \max_{i=1}^{n}\{\Phi_i\}$. For any $t \geqslant R_{max}$, the processing pattern in the intervals $[t, t + L)$ and $[t + L, t + 2 \cdot L)$ will be the same.*

Proof. Let $t = R_{max} + m \cdot L + q$, q: $0 \leqslant q < L$ and $m \geqslant 0$. For $q = 0$, we know from Lemma 5 that the processing pattern in the intervals $[t, t + L)$ and $[t + L, t + 2 \cdot L)$ will be the same.

Since, $[t, R_{max} + (m+1) \cdot L) \subset [R_{max} + m \cdot L, R_{max} + (m+1) \cdot L)$ and $[t + L, R_{max} + (m+2) \cdot L) \subset [R_{max} + (m+1) \cdot L, R_{max} + (m+2) \cdot L)$, the processing pattern in $[t, R_{max} + (m+1) \cdot L)$ and $[t + L, R_{max} + (m+2) \cdot L)$ will be the same.

Now, $[R_{max} + (m+1) \cdot L + 1, R_{max} + (m+1) \cdot L + q) \subset [R_{max} + (m+1) \cdot L, R_{max} + (m+2) \cdot L)$ and $[R_{max} + (m+2) \cdot L + 1, R_{max} + (m+2) \cdot L + q) \subset [R_{max} + (m+2) \cdot L, R_{max} + (m+3) \cdot L)$. From Lemma 5, we know that the processing pattern in $[R_{max} + (m+1) \cdot L, R_{max} + (m+2) \cdot L)$ and $[R_{max} + (m+2) \cdot L, R_{max} + (m+3) \cdot L)$ is the same hence, the processing pattern in the intervals $[R_{max} + (m+1) \cdot L + 1, R_{max} + (m+1) \cdot L + q)$ and $[R_{max} + (m+2) \cdot L + 1, R_{max} + (m+2) \cdot L + q)$ will also be the same. This implies that the processing pattern in the intervals $[t, R_{max} + (m+1) \cdot L + q)$ and $[t + L, R_{max} + (m+2) \cdot L + q)$ is also the same.

Replacing $R_{max} + m \cdot L + q$ with t, the processing pattern in $[t, t + L)$ and $[t + L, t + 2 \cdot L)$ is the same for $t \geqslant R_{max}$. □

Example. For the task set used in Lemma 3, Figs. 2(f), 3(a) and 3(b) represent the processing of all tasks in the intervals $[36, 72)$, $[72, 108)$ and $[108, 144)$ respectively. Here, $R_{max} = 0$, and $L = 36$. Consider a sample value of $t = 40$. It is easily discerned from Figs. 2(f), 3(a) and 3(b) that the processing patterns in intervals $[40, 76)$ and $[76, 112)$ are the same.

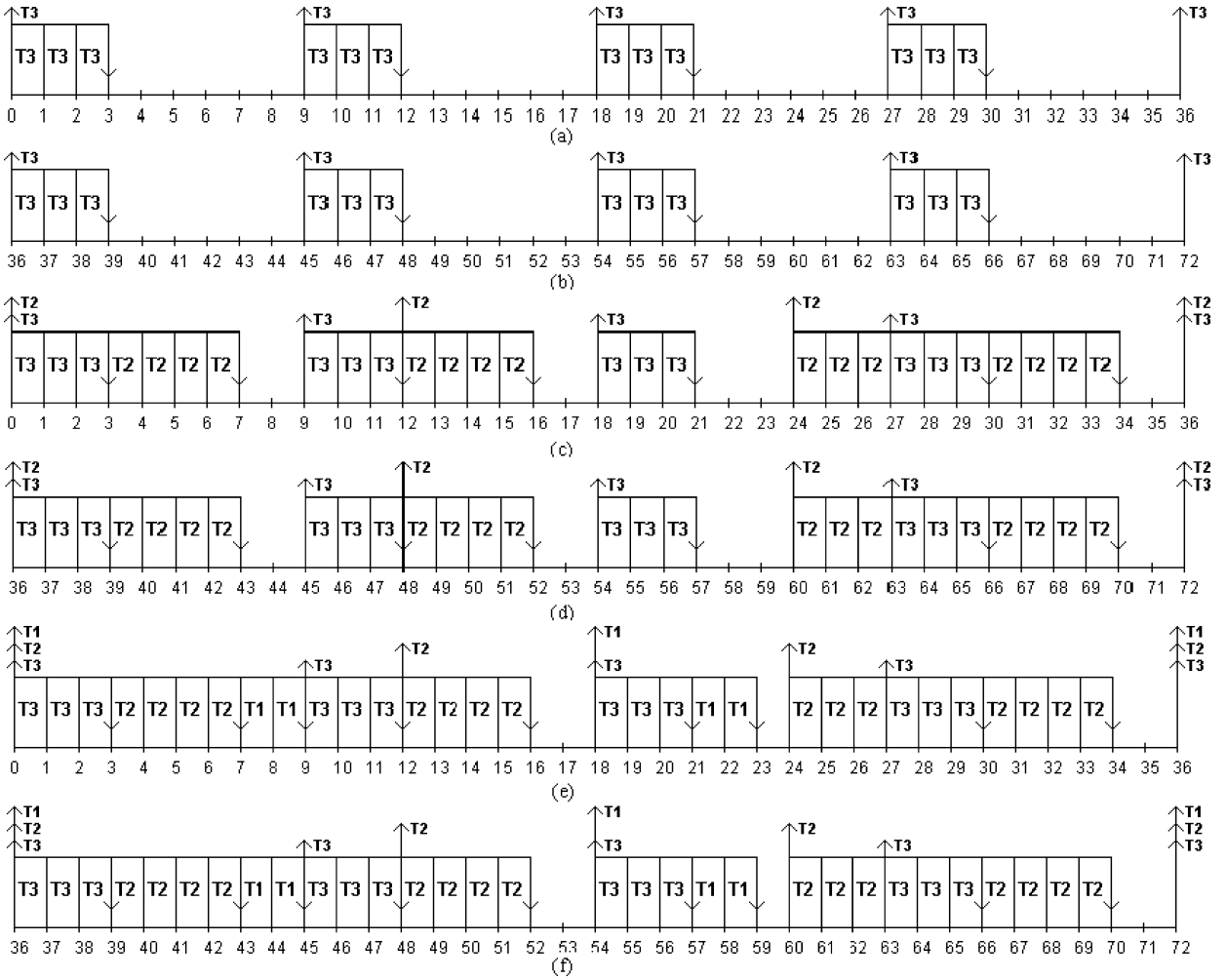


Fig. 2. Graphical representation of tables representing processing of different tasks with marked blocks. (a) Table 1 – after accounting for processing of τ_3 in the interval $[0, 36)$. (b) Table 2 – after accounting for the processing of τ_3 in the interval $[36, 72)$. (c) Table 1 – after accounting for the processing of τ_3, τ_2 in the interval $[0, 36)$. (d) Table 2 – after accounting for the processing of τ_3, τ_2 in the interval $[36, 72)$. (e) Table 1 – after accounting for the processing of τ_3, τ_2, τ_1 in the interval $[0, 36)$. (f) Table 2 – after accounting for the processing of τ_3, τ_2, τ_1 in the interval $[36, 72)$.

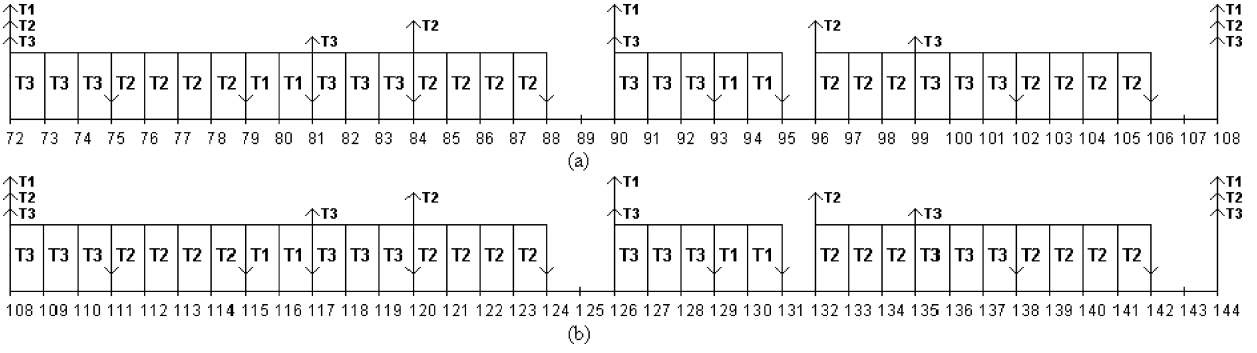


Fig. 3. Graphical representation of linear tables with marked blocks. (a) Table 1 – after accounting for processing of all tasks in the interval $[72, 108)$. (b) Table 2 – after accounting for the processing of all tasks in the interval $[108, 144)$.

Corollary 6.1. *For $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$ and $R_{max} = \max_{i=1}^{n}\{\Phi_i\}$. For any $t \geqslant R_{max}$, the processing pattern in the intervals $[t + m \cdot L, t + (m + 1) \cdot L)$ and $[t + (m + 1) \cdot L, t + (m + 2) \cdot L)$, $m \geqslant 0$, will be the same.*

Proof. For $m = 0$, and some $t_1 \geqslant R_{max}$, the processing pattern in the intervals $[t_1, t_1 + L)$ and $[t_1 + L, t_1 + 2 \cdot L)$ will be the same per Lemma 6. If $t = t_1 + L$, then $t > R_{max}$, and the processing pattern in $[t, t + L)$ and $[t + L, t + 2 \cdot L)$ will also be the same per Lemma 6. Substituting t with $t_1 + L$, we see that the processing pattern in the intervals $[t_1, t_1 + L)$, $[t_1 + L, t_1 + 2 \cdot L)$ and $[t_1 + 2 \cdot L, t_1 + 3 \cdot L)$ is the same.

This way by setting $t = t_1 + m \cdot L$: $m \geqslant 1$, we will find that the processing pattern in the intervals $[t_1, t_1 + L)$ through $[t_1 + (m + 1) \cdot L, t_1 + (m + 2) \cdot L)$ will be the same. Therefore, for $m \geqslant 0$ and $t \geqslant R_{max}$, the processing pattern in the intervals $[t + m \cdot L, t + (m + 1) \cdot L)$ and $[t + (m + 1) \cdot L, t + (m + 2) \cdot L)$ will be the same. □

Theorem 7. *For $\Gamma_n = \{\tau_1, \tau_2, \ldots, \tau_n\}$ and $R_{max} = \max_{i=1}^{n}\{\Phi_i\}$, the feasibility interval of Γ_n is $[t, t + L)$, where $t \geqslant R_{max}$.*

Proof. If a real-time system starts at time 0, and runs till time Z, where Z is a sufficiently large positive integer ($Z \to \infty$), then we can derive a number m: $m = Z \bmod L$. Hence, Z can be divided up into equal time intervals of length L. A summation operation on the consecutive time intervals of length L will lead to the time interval $[0, Z)$. Or, for any $t \geqslant R_{max}$,

$$[0, Z) = [0, t) + [t, t + L) + \cdots + \big[t + (m - 1) \cdot L, t + m \cdot L\big) + [t + m \cdot L, Z).$$

From Corollary 6.1, we know that the processing pattern in time intervals $[t, t+L), [t+L, t+2 \cdot L), \ldots, [t+(m-1) \cdot L, t+m \cdot L)$ is the same. Since $[t + m \cdot L, Z)$ is relatively small compared to $[0, Z)$, the processing in this interval can be ignored without a loss of generality. Due to the same processing pattern, if the task set is schedulable in the time interval $[t, t + L)$ it will also be schedulable in the time intervals $[t + L, t + 2 \cdot L), \ldots, [t + (m - 1) \cdot L, t + m \cdot L)$. Hence, the schedulability of the task set in the time interval $[t, t + L)$ will determine the schedulability of the task set in time intervals $[0, Z)$. Hence by its definition, the feasibility interval of Γ_n is $[t, t + L)$. □

Corollary 7.1. *The earliest feasibility interval of Γ_n is $[R_{max}, R_{max} + L)$.*

Proof. From Theorem 7, the feasibility interval of Γ_n is $[t, t + L)$, where $t \geqslant R_{max}$. The smallest possible value of t is R_{max}, hence the interval $[R_{max}, R_{max} + L)$ is the earliest possible feasibility interval. □

Corollary 7.2. *If all tasks in Γ_n are synchronously released, then the earliest feasibility interval is $[0, L)$.*

Proof. For a synchronous release $R_{max} = 0$. Setting $R_{max} = 0$ in the feasibility interval from Corollary 7.1 ($[R_{max}, R_{max} + L)$) gives $[0, L)$ as the feasibility interval. □

5. Experimental validation

To validate the results given in Theorem 7 we experimentally evaluated the feasibility interval for synthetic P-FRP task sets. 3 groups of 500 task sets, where each group had 3, 5 and 7 tasks were randomly generated. Each of the task sets used was unique in the sense that at least one task was different between two task sets. The execution time of the randomly generated tasks was selected from $[2, 20]$, while the range for arrival periods was selected from $[10, 40]$. The processor utilization factor [21] of these task sets was in the range $[0.23, 1]$. The release offsets were selected from $[1, 10]$, and the values for $t_{copy}(k)$ and $t_{restore}(k)$ were selected from $[1, 5]$. The task sets were generated and simulated using the TGSIMEx [9] system.

We computed the LCM of each task set and derived the R_{max} value for each task set. For every task set, a string representing the processing pattern was generated for the following time intervals $[R_{max}, R_{max} + L)$, $[R_{max} + L, R_{max} + 2 \cdot L)$, $[R_{max} + 2 \cdot L, R_{max} + 3 \cdot L)$, $[R_{max} + 3 \cdot L, R_{max} + 4 \cdot L)$ and $[R_{max} + 4 \cdot L, R_{max} + 5 \cdot L)$. This *processing string* is composed of task identifiers in a sequential order of processing. For example, if τ_1 is processed in the interval $[1, 4)$, τ_2 is processed in $[5, 9)$ and τ_3 is processed at times $[10, 13)$ the string representing this processing will be "1111222223333". Each of the task sets was executed in a P-FRP simulator.

Using a hash function, the processing strings for the 5 time intervals were compared and if they were different it meant that the computed feasibility interval is not correct. For everyone of the 500 task sets in each group, the processing strings for all the 5 time intervals were an exact match thus validating the feasibility interval value derived in this paper.

6. Conclusions

We have formally derived the feasibility interval in P-FRP and have shown that the fundamental attribute in deriving the feasibility interval in P-FRP is the LCM of the arrival periods of tasks, as is also the case in the preemptive model. The feasibility interval for a synchronous release in P-FRP and the preemptive model is also the same. The feasibility interval we have derived for P-FRP's asynchronous release is an improvement over Leung and Whitehead's method and simpler to derive than that of Goossens and Devillers. However, we do not claim that P-FRP's feasibility interval for asynchronous release will also work for the preemptive model, which was the focus of these previous works.

In this work, we have focused on periodic tasks. An upper bound on the feasibility interval for sporadic tasks can also be derived by considering an LCM value based on maximum possible arrival period of tasks. In ongoing work, we are developing robust response time analysis models for P-FRP using the feasibility interval derived in this paper. An exact schedulability test using the game-board method [8] to compute the actual response time is also being developed. The game-board method efficiently computes the response time of a task under a given release scenario. By computing the response time of every job that is released in the feasibility interval, we can determine the schedulability of a P-FRP task for that release scenario. Through analysis of the response time of every task in the set we can determine the schedulability of the system as long as it is in operation.

References

[1] J.H. Anderson, S. Ramamurthy, K. Jeffay, Real-time computing with lock free shared objects, ACM Trans. Comput. Syst. 5 (6) (1997) 388–395.
[2] N. Audsley, Optimal priority assignment and feasibility of static priority tasks with arbitrary start times, Technical Report, University of York, 1991.
[3] G. Bernat, Response time analysis of asynchronous real-time systems, Real-Time Syst. 25 (2003) 131–156.
[4] C. Belwal, A.M.K. Cheng, Determining actual response time in P-FRP, Technical Report: UH-CS-10-05, Dept. of Computer Science, University of Houston, 2010.
[5] C. Belwal, A.M.K. Cheng, Determining actual response time in P-FRP, in: PADL'11, 2011.
[6] C. Belwal, A.M.K. Cheng, Partitioned scheduling of P-FRP in symmetric homogeneous multiprocessors, in: IEEE/IFIP EUC'11, 2011.
[7] C. Belwal, A.M.K. Cheng, A utilization based sufficient condition for P-FRP, in: IEEE/IFIP EUC'11, 2011.
[8] C. Belwal, A.M.K. Cheng, Determining actual response time in P-FRP using idle-period game board, in: IEEE ISORC'11, 2011.
[9] C. Belwal, A.M.K. Cheng, An extensible framework for real-time task generation and simulation, in: IEEE RTCSA'11, 2011, pp. 259–263.
[10] C. Belwal, A.M.K. Cheng, W. Taha, Release offset bounds for response time analysis of P-FRP, in: IEEE ICESS'11, 2011, pp. 950–957.
[11] C. Belwal, A.M.K. Cheng, On priority assignment in P-FRP, in: RTAS'10 Work-in-Progress Session, 2010.
[12] L. Cucu, J. Goossens, Feasibility intervals for fixed-priority real-time scheduling on uniform multiprocessors, in: ETFA'06, 2006, pp. 397–405.
[13] C. Elliott, P. Hudak, Functional reactive animation, in: ICFP'97, 1997, pp. 263–273.
[14] J. Goossens, R. Devillers, The non-optimality of the monotonic priority assignments for hard real-time offset, Real-Time Syst. 13 (1997) 2.
[15] S.F. Fahmy, B. Ravindran, E.D. Jensen, Response time analysis of software transactional memory-based distributed real-time systems, in: ACM SAC Operating Systems, 2009.
[16] Haskell, http://www.haskell.org.
[17] R. Kaiabachev, W. Taha, A. Zhu, E-FRP with priorities, in: EMSOFT'07, 2007, pp. 221–230.
[18] J.Y.T. Leung, J. Whitehead, On the complexity of fixed-priority scheduling of periodic, real-time tasks, Perform. Eval. 2 (4) (1982) 237–250.
[19] J.P. Lehoczky, Fixed priority scheduling of periodic task sets with arbitrary deadlines, in: RTSS'90, 1990, pp. 201–209.
[20] C.L. Liu, L.W. Layland, Scheduling algorithms for multiprogramming in a hard-real-time environment, J. ACM 20 (1) (1973) 46–61.
[21] J. Manson, J. Baker, A. Cunei, S. Jagannathan, M. Prochazka, B. Xin, J. Vitek, Preemptible atomic regions for real-time Java, in: RTSS'05, 2005, pp. 62–71.
[22] J. Peterson, G.D. Hager, P. Hudak, A language for declarative robotic programming, in: IEEE ICRA'99, 1999.
[23] J. Peterson, P. Hudak, A. Reid, G.D. Hager, FVision: A declarative language for visual tracking, in: Symposium on Practical Aspects of Declarative Languages (PADL), 2001.
[24] J. Ras, A. Cheng, Response time analysis for the abort-and-restart task handlers of the priority-based functional reactive programming (P-FRP) paradigm, in: RTCSA'09, 2009.
[25] Z. Wan, W. Taha, P. Hudak, Real-time FRP, in: ICFP'01, ACM Press, 2001, pp. 146–156.
[26] Z. Wan, W. Taha, P. Hudak, Task driven FRP, in: PADL'02, 2002.
[27] Z. Wan, P. Hudak, Functional reactive programming from first principles, in: ACM SIGPLAN PLDI'2000, 2000, pp. 242–252.